

Lesson 7

Data Fetching & UI Update Strategies

Code the Dream — React

Today's Session

EXPLORE

- CRUD operations and HTTP methods
- Promises and the `async/await` pattern
- Fetching data inside `useEffect`
- Loading and error state
- Pessimistic vs. optimistic UI update strategies

APPLY

- Complete two fetch utility functions in `api.js`
- `FetchOnRender` — load a list of posts when the component mounts
- `FetchOnClick` — load a single post when a button is clicked

By the end: fetch data from an external API, handle loading and error states, and choose the right UI update strategy for the job.

CRUD Operations

Most app interactions with an API fall into one of four categories:

Operation	HTTP Method	Example
Create	POST	Add a new todo
Read	GET	Load a list of posts
Update	PUT / PATCH	Edit a todo's title
Delete	DELETE	Remove a todo

GET is the browser's default — no `method` needed in a `fetch` call. All others must be specified explicitly.

PUT replaces the entire record. PATCH updates only the fields you send. Most edit operations use PATCH.

The `async/await` Pattern

`async/await` makes asynchronous code read like synchronous code. Use it with a `try/catch` block to handle errors.

```
async function fetchPost(id) {  
  try {  
    const resp = await fetch(`/api/posts/${id}`);  
    if (!resp.ok) throw new Error(resp.status);  
    const data = await resp.json();  
    return data;  
  } catch (error) {  
    console.error(error.message);  
  }  
}
```

- `await` pauses the function until the promise resolves — the rest of the page stays responsive
- `resp.ok` is true for 2xx status codes — always check it before reading the body
- `throw` inside `try` jumps directly to `catch`

GET with useEffect

`useEffect` expects either no return value or a cleanup function — but `async` functions always return a promise. Solve this by declaring and immediately invoking an `async` function inside the effect.

```
useEffect(() => {  
  (async () => {  
    try {  
      const resp = await fetch('/api/posts');  
      if (!resp.ok) throw new Error(resp.status);  
      setPosts(await resp.json());  
    } catch (error) {  
      setError(error.message);  
    }  
  })();  
}, []); // empty [] → runs once on mount
```

The `()` at the end immediately invokes the `async` function. The outer arrow function (the one `useEffect` receives) is still synchronous.

Loading and Error State

Track three pieces of state for every fetch operation: the data, whether it's loading, and any error.

```
const [posts, setPosts] = useState([]);  
const [isLoading, setIsLoading] = useState(true);  
const [error, setError] = useState('');
```

Use them in the component's return to show the right thing at the right time:

```
if (isLoading) return <p>Loading...</p>;  
if (error) return <p>Something went wrong: {error}</p>;  
return <ul>{posts.map(p => <li key={p.id}>{p.title}</li>)}</ul>;
```

Set `isLoading` to `true` before the fetch starts and `false` in a `finally` block — so it always resets even if the fetch fails.

Check for Understanding

What is wrong with this code?

```
useEffect(async () => {  
  const resp = await fetch('/api/posts');  
  setPosts(await resp.json());  
}, []);
```

- A) `async` functions return a promise — `useEffect` expects `undefined` or a cleanup function, not a promise
- B) `await` can't be used inside `useEffect` at all
- C) The dependency array should list `setPosts` as a dependency
- D) `resp.json()` must be called before `await`, not after

Pessimistic vs. Optimistic UI Updates

Network requests take time. Two strategies exist for handling the gap between a user action and the server's response.

Pessimistic

Wait for the server to confirm before updating the UI.

- UI always matches server-confirmed state
- Simple to implement
- Can feel slow for frequent actions

Best for: login, payment, form submission — operations where accuracy matters more than speed

Optimistic

Update the UI immediately, sync with server in background. Roll back if the server fails.

- Feels instant to the user
- More complex error handling
- Small risk of temporary inconsistency

Best for: adding items to a cart, liking a post, reordering a list

Pessimistic Pattern

Show a loading indicator → wait for the server → update state from the response.

```
async function handleAddTodo(data) {  
  setIsLoading(true);  
  try {  
    const resp = await fetch('/api/todos', {  
      method: 'POST',  
      body: JSON.stringify(data),  
      headers: { 'Content-Type': 'application/json' },  
    });  
    if (!resp.ok) throw new Error(resp.status);  
    setTodos(prev => [...prev, await resp.json()]); // update AFTER success  
  } catch (error) {  
    setError(error.message);  
  } finally { setIsLoading(false); }  
}
```

The UI does not change until the server confirms. If the request fails, the UI stays as-is and shows an error.

Optimistic Pattern

Update the UI immediately → sync in background → roll back on failure.

```
async function handleAddToCart(item) {  
  const previousCart = cart;          // 1. save current state  
  setCart(prev => [...prev, item]);    // 2. update immediately  
  
  try {  
    await saveCartItem(item);          // 3. background API call  
  } catch (error) {  
    setCart(previousCart);              // 4. revert on failure  
    setError('Failed to save item to cart.');  }  
}
```

The rollback only works if you saved the previous state **before** the optimistic update. Do this first, every time.

Check for Understanding

A user clicks "Like" on a social media post. The like icon fills in instantly. A few seconds later, a network error shows briefly and the like icon unfills. Which strategy does this describe?

- A) Pessimistic — the UI waited for the server before showing the like
- B) Optimistic — the UI updated immediately, then rolled back when the request failed
- C) Pessimistic — the error message came from the server
- D) Optimistic — the error means the server confirmed the like was saved

Transient State — What Not to Save

Not all UI state belongs in the database. **Transient state** is temporary, UI-only state that doesn't need to be persisted to the backend.

Examples of transient state:

- Whether a dropdown is currently open
- Whether form validation errors are visible
- Which tab is active
- A loading spinner's visibility
- Sort or filter preferences applied to the current view

Ask: "Would losing this state on a page refresh be a problem?" If no — it's transient. Keep it local and don't send it to the API.

Now: Apply

Let's fetch real data from an API.

This Week's Exercise — Two Fetch Components

Open `src/exercises/lesson-07/`. You'll work in three files:

File	Your Job
<code>api.js</code>	Complete <code>getPosts()</code> and <code>getSinglePost(postId)</code>
<code>FetchOnRender.jsx</code>	Fetch a list of posts when the component mounts
<code>FetchOnClick.jsx</code>	Fetch a single post when a button is clicked

Data comes from **JSONPlaceholder** — a free mock API at <https://jsonplaceholder.typicode.com>.

You do not need to modify `studentWork.jsx` — it already imports and renders both components.

Step 1 — Complete api.js

The fetch utility functions are stubs. Fill them in.

```
const BASE_URL = 'https://jsonplaceholder.typicode.com';

export async function getPosts() {
  const resp = await fetch(`${BASE_URL}/posts`);
  if (!resp.ok) throw new Error(resp.status);
  return resp.json();
}

export async function getSinglePost(postId) {
  const resp = await fetch(`${BASE_URL}/posts/${postId}`);
  if (!resp.ok) throw new Error(resp.status);
  return resp.json();
}
```

Both functions throw on a bad response — the calling component's `catch` block will handle the error.

Step 2 — FetchOnRender

Call `getPosts()` inside a `useEffect` and display the results.

State + Effect

```
const [posts, setPosts] = useState([]);

useEffect(() => {
  (async () => {
    try {
      const data = await getPosts();
      setPosts(data);
    } catch (error) {
      console.error(error);
    }
  })();
}, []);
```

JSX

```
return (
  <ul>
    {posts.map(post => (
      <li key={post.id}>
        <h2>{post.title}</h2>
        <p>{post.body}</p>
      </li>
    ))}
  </ul>
);
```


Step 3 — FetchOnClick

Call `getSinglePost()` when the button is clicked — no `useEffect` needed.

```
const [post, setPost] = useState(null);

async function handleFetchPost() {
  try {
    const data = await getSinglePost(1);
    setPost(data);
  } catch (error) {
    console.error(error);
  }
}

<button onClick={handleFetchPost}>Get post</button>
{post && (
  <div>
    <h2>{post.title}</h2>
    <p>{post.body}</p>
  </div>
)}
```

Stretch Goals

If you finish early, add these improvements to both components:

Loading state

```
const [isLoading, setIsLoading] = useState(false);  
// setIsLoading(true) before fetch, setIsLoading(false) in finally  
{isLoading && <p>Loading...</p>}
```

Error handling

```
const [error, setError] = useState('');  
// setError(error.message) in catch  
{error && <p>Error: {error}</p>}
```

Limit results — add a query param to `getPosts()` in `api.js`:

```
fetch(`${BASE_URL}/posts?_limit=10`)
```

Wrap-Up

This week you learned:

- CRUD maps to HTTP methods: GET, POST, PUT/PATCH, DELETE
- Use `async/await` with `try/catch` — always check `resp.ok` before reading the body
- Fetch inside `useEffect` using an IIFE — never pass `async` directly to `useEffect`
- Track loading and error state for every fetch operation
- Pessimistic: wait for the server; Optimistic: update now, revert on failure

Before next session: Complete the weekly assignment — connect your todo app to the CTD API with authentication, data fetching, and optimistic updates for add/complete/edit. Requires environment setup — start early. Submit your PR link in CTD Learns. Post questions in **`#[class-name]-discussion`** on Slack.